A Sample Article Using IEEEtran.cls for IEEE Journals and Transactions

IEEE Publication Technology, *Staff, IEEE,*

***Abstract*— Coordinating task allocation and planning for a heterogeneous multi-robot system is very challenging under communication and temporal logic constraints. To address this, we develop a decomposition-based semi-distributed task allocation and planning framework (D^2TAPR). Its semi-distributed nature arises from leader-based coordination during collaborative subtasks. The global task specified in linear temporal logic (LTL) is translated into a pruned Buchi automaton and ¨ decomposed into subtasks. Optional subtasks are planned via a consensus-based timetable algorithm. Collaborative subtasks requiring multiple robots are handled via leader-based coordination with timeout-triggered recruitment, while failed subtasks are reassigned through a free-market-based mechanism, which designates a region called free market for information exchange and subtask redistribution. A patrol mechanism is triggered when prerequisite subtasks required by temporal logic constraints become overdue, gathering information to prevent task blocking. Experiments on 25 benchmarks show that our approach is efficient and reliable.**

***Index Terms*—Article submission, IEEE, IEEEtran, journal, LATEX, paper, template, typesetting.**

I. Introduction

Multi-robot systems (MRS) are increasingly important in applications such as patrol, surveillance, and logistics [1] [2], because teams can deliver higher efficiency, robustness, and flexibility than a single robot. Task allocation and planning is a core problem in MRS and has attracted widespread attention. Existing methods are generally divided into centralized and distributed algorithms. Centralized algorithms [3] [4] rely on a coordinator with global information to compute globally optimal assignments; however, they incur high computational costs and are vulnerable to failures, which limits scalability and robustness in dynamic environments. Distributed algorithms allow robots to make decisions based on local information and limited communication, including auction algorithms [5] [6], contract net protocols [7], and rule-based methods [8]. Distributed algorithms demonstrate significant application potential in dynamic scenarios with constrained communication, such as security patrols, post-disaster search and rescue [9].

In practical applications, many multi-robot tasks include temporal logic constraints, such as ordering, persistence, and synchronization requirements [10]. For example, in industrial inspection of tunnels or pipelines, robots may be required to survey regions in a prescribed order to respect safety or access dependencies (e.g., a valve or passage must be verified before downstream areas are entered). To specify these requirements precisely, recent studies have increasingly adopted formal languages [11], among which linear temporal logic (LTL) is a widely used tool [12]. LTL enables unambiguous task specification.

Under LTL constraints, existing task allocation and planning methods are typically categorized into top-down and bottom-up approaches. In top-down methods, a global LTL formula specifies the overall mission, and a centralized planner decomposes this specification into robot-level plans. A common approach constructs a product automaton from a global LTL-derived nondeterministic B"uchi automaton (NBA) and robot discrete transition systems, and then solves the resulting graph problem [13], [14]. However, state explosion caused by increased robot numbers, larger environments, and more complex tasks limits this method's scalability in large systems. To address this limitation, sampling-based [15], mixed-integer programming-based methods [16], and incremental decision tree-based methods [17] have been proposed to reduce computational complexity. Sampling-based methods [15], [18] reduce computational load by constructing generation trees from NBA samples rather than the full product automaton; however, they typically pre-specify the executing robot team for each subtask (or a set of candidate teams), which limits flexibility. Chen et al. [17], [19] proposed an incremental decision tree-based method that significantly reduces computation, but it can be prone to locally optimal solutions. Recognizing the parallel and concurrent nature of MRS, some studies decompose a global specification into multiple subtasks using automaton transition relations to increase parallelism. Schillinger et al. [20], [21] decompose the global task into a set of parallel subtasks. Virtual transition edges are then used to manage transitions between these parallel subtasks, enhancing system parallelism; however, this method does not explicitly handle collaborative subtasks. Based on these works, Liu et al. [22], [23] further improved efficiency by representing subtask temporal relations with posets and solving via a branch-and-bound method. Recent studies also combine large language models with optimization-based schedulers, such as DEXTER-LLM [24]. Overall, top-down approaches are almost always centralized and depend on a central coordinator with global information, which makes them hard to deploy under limited communication.

Bottom-up approaches, in contrast, assume each robot is assigned a local LTL specification, and robots then coordinate through distributed planning and information exchange. From this viewpoint, temporal logic planning becomes a task coordination problem. Filippidis et al. [25] proposed a distributed planning framework under temporal logic constraints enables agents to exchange information and verify conflicts online.

This paper was produced by the IEEE Publication Technology Group. They are in Piscataway, NJ.

Manuscript received April 19, 2021; revised August 16, 2021.

Each agent can communicate only with neighbors within its communication range, and a dedicated "follower" agent is used to establish and maintain communication links when interaction is required. In [26], [27], each robot updates its knowledge based on local observations and propagates it through neighbor communication; with newly received knowledge, robots confirm and adjust their action paths online. However, because such neighbor-only communication cannot guarantee that all robots required for a collaborative subtask can mutually communicate when needed, these methods do not handle collaborative tasks effectively. For scenarios with collaborative tasks, Tian et al. [28] proposed a distributed cooperative computation framework that decomposes task constraints via an interaction-edge product automaton, enabling agents to synthesize local trajectories in parallel using only neighborhood interaction information. Most work focuses on avoiding physical collisions while overlooking task conflicts, Chen et al. [29] address task-level conflict resolution under communication constraints by predicting goals of robots beyond communication range. While these approaches are more scalable and robust, their key assumption that local tasks are known or preassigned limits applicability in many real-world scenarios.

These limitations become more pronounced when operators specify only a global mission rather than explicit local tasks. A representative setting is multi-robot inspection or patrol in large industrial facilities, photovoltaic power stations and so on, where communication propagation restricts robots to exchange information only with neighbors within a fixed range. In such settings, high-level requirements such as "regions must be visited in a prescribed order while avoiding conflicts" are naturally expressed as global LTL formulas, whereas preassigning local tasks to individual robots is nontrivial and can be brittle under robot failures. Moreover, centralized planning becomes difficult when communication is range-limited and a single coordinator is a potential point of failure. Therefore, distributed solutions for global LTL missions under limited communication range are required. However, under limited communication, distributed execution faces two distinct issues. First, task blocking arises from temporal logic constraints: a robot may not obtain the prerequisite completion information needed to start its subtask. Second, collaborative subtasks may fail when required robots do not arrive on time, and consecutive collaboration failures can cause large delays in overall completion time. Common strategies mitigate such failures by reassigning vacant roles to communicable robots, reprioritizing tasks, or reissuing subtasks [30]–[32], but their resilience is limited.

Motivated by these challenges, this work develops a semi-distributed task allocation and planning framework under temporal logic and communication constraints, where the "semi-distributed" nature arises from leader-based coordination during collaborative subtasks. Specifically, to address task blocking under limited communication because of the temopral constrants, a patrol mechanism that proactively gathers information to release blocked subtasks was proposed; to address collaboration failures, a resilient collaboration mechanism that combines leader-based coordination with timeout-triggered recruitment and free-market-based reassignment was proposed to mitigate (i) collaborative subtask failures caused by robots failing to arrive at required locations on time and (ii) large completion-time delays caused by consecutive subtask failures. Here, the free market is a designated rendezvous region where robots exchange state information and handle failed subtasks. The main contributions are as follows:

- A decomposition-based semi-distributed task allocation and planning framework with resilient collaborative execution (D^2TAPR) is proposed under communication constraints, offering efficient and robust task completion. Theoretical analysis shows that, given the assumptions in this paper, the task can be successfully completed.
- To address task blocking under limited communication, a patrol mechanism is proposed to actively gather information when prerequisite subtasks are overdue but completion is unconfirmed.
- To address collaboration failures, two complementary mechanisms are designed: a leader-based coordination with timeout-triggered recruitment to reduce subtask failures, and a free-market-based reassignment to mitigate significant delays when failures occur.

Notations: Let $\mathbb{N}$ and $[N]$ denote the set of natural numbers and the shorthand notation for $\{1, \ldots, N\}$, respectively. Given a set A, let $|A|$ and 2^A denote the cardinality and power set of A, respectively.

II. PRELIMINARIES

LTL formulas are constructed from atomic propositions AP using Boolean operators and temporal operators. Atomic propositions represent basic Boolean variables whose values are evaluated as either true or false. Detailed content can be found in [12]. LTL formulas can be translated into NBA.

Definition 1 (Nondeterministic Büchi Automaton)**.** *A nondeterministic Büchi automaton (NBA) is a tuple $\mathcal{B} = (S, S_0, \Sigma, \delta, F)$, where S is a finite set of states, $S_0 \subseteq S$ is the set of initial states, $\Sigma = 2^{AP}$ is the input alphabet, $\delta : S \times \Sigma \to 2^S$ is the transition function, and $F \subseteq S$ is the set of accepting states.*

Given an infinite word $\omega = \sigma_0\sigma_1\sigma_2\ldots$ with $\sigma_i \in \Sigma$, a run $\rho = s_0 s_1 s_2 \ldots$ of the NBA $\mathcal{B}$ over ω is an infinite state sequence such that $s_0 \in S_0$ and $s_{i+1} \in \delta(s_i, \sigma_i)$ for all $i \geq 0$. The run ρ is accepting if it visits states in F infinitely often. Accepting runs can be represented in a prefix–suffix structure, where the prefix w_{pre} runs from an initial state to an accepting state, and the suffix w_{suf} forms a cycle containing at least one accepting state.

This paper focuses on a special class of LTL formulas called syntactically co-safe LTL (sc-LTL) [33]. sc-LTL formulas restrict temporal operators to $\bigcirc$ (next), $\cup$ (until), and $\Diamond$ (eventually), and must be written in a positive normal form.

III. PROBLEM STATEMENT

A. Environment and Multi-Robot System

In a workspace $\mathcal{W} \subseteq \mathbb{R}^2$, let $\mathcal{L} = \{l_1, l_2, \ldots, l_M\}$ be M pairwise disjoint target regions, i.e., $l_i \cap l_j = \emptyset$ for all $i \neq j$,

where $i, j \in [M]$. Let $\mathcal{O}$ denote uninteresting regions, such as forbidden areas, and assume $l_i \cap \mathcal{O} = \emptyset$.

Given workspace $\mathcal{W}$, we consider a heterogeneous robot team $\mathcal{R} = \{r_1, r_2, \ldots, r_N\}$, where N is the number of robots. The robots have n_t different types, where each robot belongs to exactly one type. Let $\mathcal{T} = [n_t]$ denote the type index set. For each $i \in \mathcal{T}$, let $\mathcal{R}_i \subseteq \mathcal{R}$ denote the set of robots of type i. Then $\mathcal{R} = \bigcup_{i=1}^{n_t} \mathcal{R}_i$ and $\mathcal{R}_i \cap \mathcal{R}_j = \emptyset$, $\forall i \neq j$. The velocity and position of robot r_k are denoted by v_k and x_k, respectively.

Consider a time sequence $\{\tau_n = n\Delta t\}_{n\in\mathbb{N}}$ with sampling interval Δt. For convenience, for any quantity y, $y(\tau_n)$ denotes the value or state of y at time τ_n. In other words, the argument τ_n indicates the realization of the corresponding variable evaluated at τ_n.

Assuming communication range $D \in \mathbb{R}^+$, robots can only communicate with neighbors within this range. The neighbor set of robot r_k at time τ_n is defined as: $\mathcal{N}_k(\tau_n) = \{r_j \mid \|x_k(\tau_n) - x_j(\tau_n)\| \leq D\}$. The communication frequency is denoted by f_{com} Hz.

B. Problem Description

To better describe the problem, we first introduce the atomic propositions. Let $a^m_{i,w}$ represent the action required for task completion, indicating that w robots of type i must execute this action at region $l_m \in \mathcal{L}$, where $i \in \mathcal{T}$, $w \in \mathbb{N}$ with $w \leq |\mathcal{R}_i|$. The following atomic propositions can then be introduced: i) The atomic proposition p_m is true when any robot is located in region l_m, and false otherwise; ii) The atomic proposition $l^m_{i,w}$ is true iff action $a^m_{i,w}$ has been completed.

To facilitate task allocation and planning under temporal logic constraints, a task model was introduced to explicitly relate subtasks, actions and workspace information. This abstraction provides a unified representation for subsequent planning and scheduling. Based on this, a task model is constructed to describe the subtasks.

Definition 2 (Task Model)**.** *The task model is defined as* $TM = (\mathcal{Q}, \mathcal{A}, \mathcal{W}, \text{LA}, \text{LC}, \text{LR}, \text{LW})$*, where* $\mathcal{Q} := \{q_i\} \cup \{\varepsilon_q\}$ *is the set of subtasks and* ε_q *is the null subtask;* $\mathcal{A} := \{a^m_{i,w} \mid m \in [M],\ i \in [n_t],\ w \in [|\mathcal{R}_i|]\} \cup \{\varepsilon_a\}$*, where* ε_a *is the empty action;* $\mathcal{W}$ *is the workspace;* $\text{LA} : \mathcal{Q} \mapsto \mathcal{A}$ *associates subtasks with actions;* $\text{LC} : \mathcal{A} \mapsto \mathbb{R}_{\geq 0}$ *indicates the cost required to execute an action;* $\text{LR} : \mathcal{A} \mapsto \mathbb{N}$ *specifies the number of robots required to execute an action; and* $\text{LW} : \mathcal{A} \mapsto \mathcal{W}$ *specifies the target region for an action (excluding* ε_a*).*

Given workspace $\mathcal{W}$ and NBA $\mathcal{B}$ corresponding to the LTL-specified global task φ, the plan for robot r_k is defined as a subtask-action-time sequence $\Pi_k = (q_{0,k}, a_{0,k}, t_{0,k})(q_{1,k}, a_{1,k}, t_{1,k}) \ldots (q_{h_k,k}, a_{h_k,k}, t_{h_k,k})$, where $q_{0,k} = \varepsilon_q$, $a_{0,k} = \varepsilon_a$, and $q_{i,k} \in \mathcal{Q}$ denotes the i-th subtask of robot r_k in Π_k; $a_{i,k} \in \mathcal{A}$ represents the action taken by r_k to complete subtask $q_{i,k}$; $t_{i,k}$ indicates the time at which r_k starts executing action $a_{i,k}$. h_k denotes the length of the plan. The notation $\Pi_{i,k}$ refers to the i-th tuple in Π_k.

Example 1. *Consider a heterogeneous robot group* $\mathcal{R} = \{r_1, r_2, r_3, r_4\}$*, including two type-1 (e.g., inspection) and two type-2 (e.g., maintenance and manipulation) robots, performing distributed tasks in a large-scale industrial park. The target regions are* $\mathcal{L} = \{l_1, l_2, l_3, l_4\}$*, as shown in Fig. 4. The task requires two type-1 robots to go to* l_1 *for a joint inspection of an abnormal facility status, and one type-2 robot must travel to a local service station at* l_2 *to perform an on-site handling operation. In addition, one type-1 and one type-2 robot must go to* l_3 *for a coordinated repair task. During this coordinated-repair period, no robot may pass through region* l_4*, which represents a hazardous area. Let* p_4 *denote the atomic proposition that any robot enters region* l_4*. The task can be described using the following sc-LTL formula:*

$$\varphi = \Diamond l^1_{1,2} \wedge \Diamond l^2_{2,1} \wedge \Diamond (l^3_{1,1} \wedge l^3_{2,1} \wedge \neg p_4) \tag{1}$$

Based on the above, the workspace schematic is shown in Fig. 4. The problem studied in this work is described as follows.

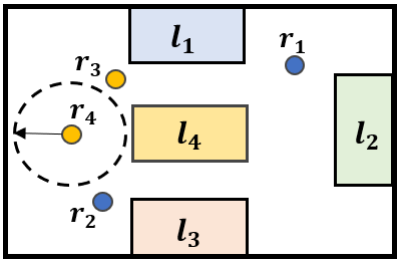

Fig. 1. Schematic of the workspace. The blue circles denote type-1 robots, and the yellow circles denote type-2 robots.

Problem 1. *Given a global sc-LTL task specification* φ*, workspace* $\mathcal{W}$*, and a team of heterogeneous robots* $\mathcal{R}$*, the goal is to develop a distributed task-allocation and planning strategy* Π_k *under limited communication range for each robot* $r_k \in \mathcal{R}$ *such that the overall plan* Π *satisfies the task* φ*.*

The following assumptions are made to facilitate the work:

i) robots can detect the completion of a subtask upon arriving at the associated target region;

ii) any robot within the broadcast range of a sender (another robot) will instantly receive its messages;

iii) the robot team contains sufficient robots.

IV. D^{2}TAPR ALGORITHM

To make the algorithm description clearer, this section first provides a high-level overview and then presents the key modules of the proposed D^{2}TAPR algorithm in detail.

A. Overview

As shown in Fig. 2, the overall workflow has two stages. In stage 1 (task preprocessing), the global sc-LTL specification is converted into an NBA, and the NBA is then pruned and decomposed to obtain executable subtasks and their temporal dependencies. In stage 2 (task allocation and planning), robots assign subtasks via a consensus-based timetable algorithm(CBTA) under limited communication, with additional handling for collaborative subtasks and a patrol mechanism for prerequisite-information collection.

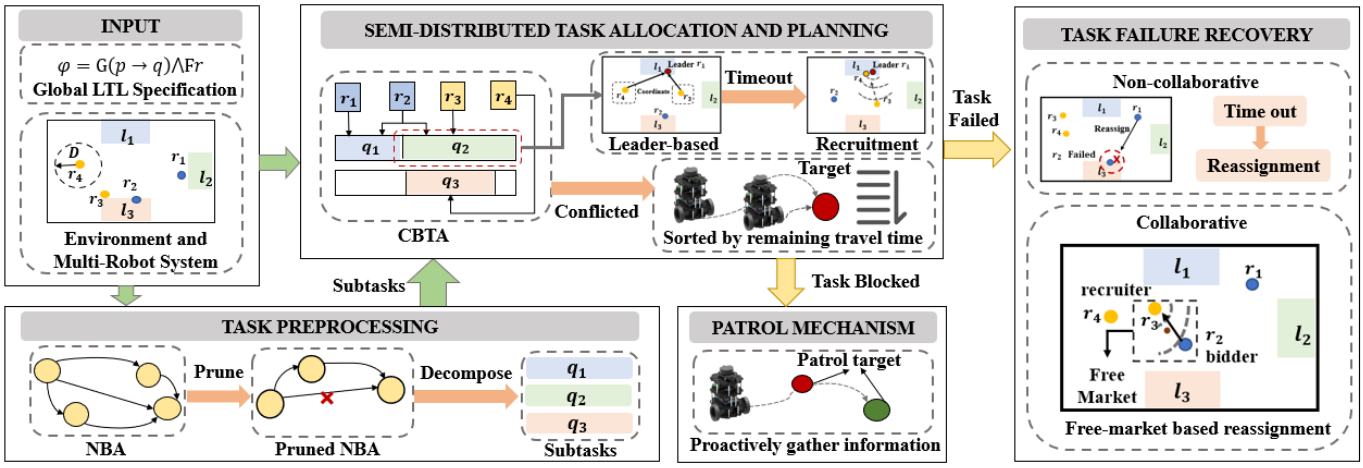

Fig. 2. Framework of D^{2}TAPR. The algorithm mainly includes two stages: task preprocessing and semi-distributed task allocation and planning.

For each collaborative subtask, the first robot reaching the target region becomes a temporary leader. The leader waits until the collaboration waiting time expires; if the arrived team still does not meet the required robot types and team size, it first recruits robots reachable through direct or multi-hop communication. If the team is still incomplete, it checks whether current arrivals, together with reachable robots and robots in the free market, satisfy the required robot types and team size. If so, the leader sends the nearest recruiter to the free market; otherwise, the subtask is declared failed. The subtask is also declared failed if recruitment is not completed within the recruiter timeout. Failed collaborative subtasks are reassigned in the free market based on estimated task completion time and task priority.

To prevent robots from getting stuck while waiting for information indicating that the remaining tasks can begin execution, the patrol mechanism lets robots actively patrol once prerequisite completion remains unconfirmed beyond the estimated completion time, so blocked subtasks can be released and execution can continue.

B. Task Preprocessing

To address Problem 1, we preprocess φ. As shown in Fig.2, preprocessing has two phases: i) NBA pruning; and ii) task decomposition on the pruned NBA to obtain the subtask set $\mathcal{Q}$ and the temporal relations among subtasks.

1) Pruning of NBA: The LTL formula is first converted into an NBA $\mathcal{B}$ and then pruned to remove redundancies, yielding a simplified automaton $\mathcal{B}^-$, as illustrated in Fig. 3. The pruning operations are:

- *Removal of infeasible transitions:* removing transitions that cannot be realized by the robot team, e.g., when the required number or types of robots exceed those available.
- *Elimination of decomposable transitions:* Decomposable transitions are removed to improve task decomposition effectiveness [34].

The definition of a decomposable transition is given in definition 3.

Definition 3 (Decomposable Transition)**.** *In NBA $\mathcal{B}$, a transition from state s_i to s_j is called decomposable if there exists another state s_k such that, for any $\sigma_{ik}, \sigma_{kj} \subseteq \Sigma$ satisfying $s_k \in \delta(s_i, \sigma_{ik})$ and $s_j \in \delta(s_k, \sigma_{kj})$, it holds that $s_j \in \delta(s_i, \sigma_{ik} \cup \sigma_{kj})$.*

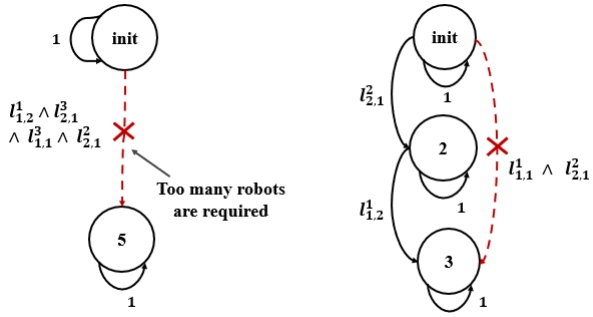

Fig. 3. Pruning of the NBA automaton: (a) pruning infeasible transitions, illustrated by a case where the required number of robots is too large; (b) pruning decomposable transitions.

Lemma 1: For any feasible word w accepted by $\mathcal{B}$, there exists a corresponding word w' that is accepted by $\mathcal{B}^-$.

Proof: Consider an accepted word $w = \cdots\{\sigma_n\}\cdots$ with accepting run $\rho = \cdots s_i s_j \cdots$ in $\mathcal{B}$, where $s_j \in \delta(s_i, \sigma_n)$. The first pruning step removes only infeasible transitions, so every feasible accepted word is preserved. For the second pruning step, consider two cases. For Case 1,if ρ contains no decomposable transitions, then all transitions of ρ remain in $\mathcal{B}^-$; hence ρ is also an accepting run in $\mathcal{B}^-$ and we can set $w' = w$. For Case 2, if a transition from s_i to s_j is removed as decomposable, there exists a state s_k and labels σ_{ik}, σ_{kj} such that $s_k \in \delta(s_i, \sigma_{ik})$, $s_j \in \delta(s_k, \sigma_{kj})$, and $\sigma_{ik} \cup \sigma_{kj} = \sigma_n$. Therefore, the transition $s_i \rightarrow s_j$ can be replaced by two transitions, $s_i \rightarrow s_k$ and $s_k \rightarrow s_j$. If both transitions $s_i \rightarrow s_k$ and $s_k \rightarrow s_j$ are non-decomposable, this reduces to Case 1. Otherwise, the same decomposition argument is applied recursively. Because a transition cannot be decomposed indefinitely into realizable finite expressions,

this recursive process terminates. The final constructed run has no decomposable transitions, so it reduces to the first case and is accepted by $\mathcal{B}^-$.

2) Task Decomposition: Following [35], $\mathcal{B}^-$ is decomposed into subtasks and a relaxed partial order is extracted. A depth-first search on $\mathcal{B}^-$ first generates an accepting path and its corresponding task sequence. Starting from this totally ordered sequence, adjacent subtasks are iteratively swapped and each new path is checked for acceptability. This process relaxes unnecessary temporal constraints and yields a logically consistent subtask set that supports parallel execution, together with a relaxed partial order.

Example 2. *For the task in Example 1, the set of subtasks corresponding to φ is given by $\Omega_\varphi = \{q_1, q_2, q_3\}$, where $q_1 = \{1, \{l^1_{1,2}\}, \{\}\}, q_2 = \{2, \{l^3_{2,1}\}, \{\}\}, q_3 = \{3, \{l^3_{1,1}, l^3_{2,1}\}, \{p_4\}\}$. Here, each subtask is represented by three entries: the first entry (e.g., 1, 2, 3) is the subtask index, the second set contains the actions to be executed, and the third set contains atomic propositions required on the self-loop.*

C. Semi-Distributed Task Allocation and Planning

This section describes how feasible plans are generated under limited communication, and how these plans are executed through real-time conflict detection and resolution as well as leader-based coordination for collaborative subtasks.

1) Consensus-based Timetable Algorithm: To prevent robots from violating temporal-order constraints, the optional subtask set $\mathrm{F}_k(\tau_n)$ for robot r_k at time τ_n is defined.

Definition 4 (Optional Subtask Set)**.** *At time τ_n, the optional subtask set $\mathrm{F}_k(\tau_n)$ for robot r_k includes all subtasks $q_i \in \mathcal{Q}$ that satisfy: i) the numbers or types of robots committed to q_i have not yet met the task requirements; ii) r_k is unassigned and capable of performing q_i; iii) temporal constraints are satisfied.*

Note that robots committed to q_i include those traveling to, waiting in, or executing the subtask at the target region. To reduce failures in collaborative subtasks, a robot commits only when its currently available information indicates that enough planned participants, in both number and required types, are already allocated to that subtask.

Allocation within $\mathrm{F}_k(\tau_n)$ is performed using CBTA [5], which can efficiently allocate tasks by iterative timetable construction and consensus-reaching among robots. The cost of the current plan for robot r_k, i.e., the plan completion time, is defined as

$$\mathrm{Cost}(\Pi_k(\tau_n)) = t_{h_k(\tau_n),k} - t_{0,k} + \mathrm{LC}(a_{h_k(\tau_n),k}). \tag{2}$$

Here, $h_k(\tau_n)$ is the current plan length of robot r_k at time τ_n; $t_{h_k(\tau_n),k}$ is the start time of the last action in $\Pi_k(\tau_n)$; $t_{0,k}$ is the start time of the plan; and $\mathrm{LC}(a_{h_k(\tau_n),k})$ is the execution cost of the last action. For each $q_i \in \mathrm{F}_k(\tau_n)$ and action $a_j \in \mathrm{LA}(q_i)$, candidate insertion positions after $\Pi_{0,k}(\tau_n)$ are evaluated by minimizing marginal cost, where $\Pi_{0,k}(\tau_n)$ is the initial anchor tuple of the plan for robot r_k at time τ_n.

$t_{i,k} \in \Pi_{i,k}(\tau_n)$ is computed as:

$$t_{i,k} = \begin{cases} t_{0,k} + \dfrac{t_{\mathrm{left},k} + d_{x_k,x_{i,k}}}{v_k}, & \text{if } i = 1 \\ t_{i-1,k} + \mathrm{LC}(a_{i-1,k}) + \dfrac{d_{x_{i-1,k},x_{i,k}}}{v_k}, & \text{otherwise} \end{cases} \tag{3}$$

where $t_{\mathrm{left},k}$ is the remaining time for the current subtask; $d_{x_k,x_{i,k}}$ is the distance from $x_k(\tau_n)$ to the target region of $q_{i,k}$; and $d_{x_{i-1,k},x_{i,k}}$ is the distance between the target regions of $q_{i-1,k}$ and $q_{i,k}$.

To enforce robot-quantity requirements, Eq. (2) is augmented with a penalty $M \sum_{i \in [h_k(\tau_n)]} L_{i,k}$, where $L_{i,k}$ is the number of robots still required to accomplish subtask q_i and M is a large constant.

Assume $P_k(\tau_n)$ is the set of plans received by robot r_k at time τ_n. Conflict resolution is then performed based on these shared plans as follows. For each candidate subtask $q_i \in \mathrm{F}_k(\tau_n)$ and each action $a_j \in \mathrm{LA}(q_i)$, robot r_k first counts how many robots are currently assigned to execute a_j according to $P_k(\tau_n)$. Let $R_j^{i,k}$ denote the number of robots that r_k still considers necessary for a_j. If the assigned number is greater than $R_j^{i,k}$, an over-assignment conflict is detected; in this case, only the $R_j^{i,k}$ robots with the earliest expected completion times are kept, because this choice minimizes the global completion cost $\mathrm{Cost_g}(\tau_n)$. If the assigned number does not exceed $R_j^{i,k}$, no conflict exists and all currently assigned robots are retained. See [5] for algorithmic details. To be noticed, the CBTA is executed in an event-driven manner. Specifically, replanning is triggered by concrete state updates, including the reception of new state information from neighboring robots, task completion, detection of over-assignment conflicts, and failure events in subtasks, which helps reduce unnecessary computation and avoids oscillations under limited communication.

Example 3. *For robot r_1 in Example 1, suppose its optional subtask set is $\mathrm{F}_1(\tau_n) = \{q_1, q_3\}$ and the initial plan is $\Pi_1(\tau_n) = (\varepsilon_q, \varepsilon_a, \tau_n)$, where ε_q and ε_a denote empty subtask and action, respectively. The actions of robot r_1 for q_1 and q_3 are $a^1_{1,2}$ and $a^3_{1,1}$ with $\mathrm{LC}(a^1_{1,2}) = 2.6$ and $\mathrm{LC}(a^3_{1,1}) = 2.5$, where 2.6 and 2.5 denote the time required to complete the corresponding actions. First, $a^1_{1,2}$ is inserted; at this stage it can only be placed after $\Pi_{0,1}$, yielding $\Pi_1(\tau_n) = (\varepsilon_q, \varepsilon_a, \tau_n)(q_1, a^1_{1,2}, \tau_n + 5.2)$. After that, $a^3_{1,1}$ is inserted; at this stage, it can be inserted either between $\Pi_{0,1}$ and $\Pi_{1,1}$ or after $\Pi_{1,1}$, and the corresponding marginal costs are 16.7 and 7.9, respectively. Then, $\Pi_1(\tau_n)$ is further updated to $(\varepsilon_q, \varepsilon_a, \tau_n)(q_1, a^1_{1,2}, \tau_n + 5.2)(q_2, a^3_{1,1}, \tau_n + 13.2)$.*

2) Real-time Conflict Detection and Resolution: During plan execution, limited communication can cause task conflicts, so a conflict detection and resolution mechanism is implemented [1].

To support this process, each robot classifies subtasks into three types at time τ_n. i) Unassigned subtasks: subtasks for which the currently committed robots are still insufficient in number or type, so additional robots may still join. ii) Completed subtasks: subtasks whose required actions have

already been executed and therefore no longer need allocation. iii) Confirmed subtasks: subtasks whose current commitments have been checked and verified to satisfy all participation requirements, meaning they are ready for execution or being executed under the current state . Let $T^k_{\text{not}}(\tau_n)$ and $T^k_{\text{already}}(\tau_n)$ denote the unassigned-subtask set and completed-subtask set maintained by robot r_k, respectively. During communication, robots exchange these sets and merge newly received updates. In this way, the optional subtask set is maintained online and remains consistent with the latest distributed task state.

Each robot r_k maintains and periodically broadcasts $S_k(\tau_n)$, i.e., the robot-state information set collected by r_k at time τ_n. In particular, each robot broadcasts not only its own state but also the peer states it has received, enabling multi-hop information propagation. Formally, $S_k(\tau_n) = \{S_{m,k}(\tau_n) \mid r_m \in \mathcal{R}, m \neq k\}$, and each element is defined as $S_{m,k}(\tau_n) = \{\tau_m, q_m, a_m, t^m_a, t^m_s, p_m, \Pi_m(\tau_n)\}$, where each element records the latest status of robot r_m as received by r_k at time τ_n. Specifically, τ_m is the generation timestamp of r_m's latest status message; q_m is the subtask currently committed by r_m; $a_m \in \text{LA}(q_m)$ is the concrete action role that r_m undertakes for q_m; t^m_a is the estimated remaining travel time from r_m's current estimated position to the target region of q_m; t^m_s is the estimated task start time. Let $\mathcal{A}_m$ denote the set of estimated arrival times of other robots assigned to the same task region; then t^m_s is defined as the difference of r_m's own estimated arrival time and the $\text{LR}(a_m)$-th smallest element in $\mathcal{A}_m$ and is always non-negative. p_m is the position of r_m; and $\Pi_m(\tau_n)$ is the plan of robot r_m at time τ_n.

Using the robot-state information set $S_k(\tau_n)$, the unassigned subtask set $T^k_{\text{not}}(\tau_n)$, and the completed subtask set $T^k_{\text{already}}(\tau_n)$, robot r_k periodically checks for conflicts while executing $a_j \in \text{LA}(q_i)$: i) Conflict caused by task completion: At each check, r_k verifies whether q_i has entered $T^k_{\text{already}}(\tau_n)$. If yes, the current commitment is immediately invalidated because further motion for q_i is redundant and may delay other pending tasks. Robot r_k then stops and replans. ii) Over-assignment: If the number of robots committed to a_j exceeds the required cardinality $\text{LR}(a_j)$, robot r_k ranks all committed robots by their remaining travel time to the target region (smallest t^m_a first) and keeps only the first $\text{LR}(a_j)$ robots as valid executors of a_j. If multiple robots have the same t^m_a, the tie is broken by robot ID in ascending order. If r_k is not selected after this filtering step, it immediately stops the current commitment and replans.

3) Free Market: The free market is a designated rendezvous place for two purposes: global information synchronization and failed-subtask reassignment. Robots enter this region in three cases: i) they currently have no executable pending subtask (unless all subtasks are already finished); ii) their assigned optional subtask cannot proceed because of failure or missing collaborators; and iii) they are explicitly dispatched by a leader as emergency recruiters. Robots broadcast the ID set of robots $\mathcal{R}_f$ currently in the free market and the corresponding timestamp, and then updates its locally maintained free-market ID set $\mathcal{R}_f$ according to the newest received timestamp.

The free-market location is computed using entropy-based weighting of subtask information. For each subtask $q_i \in \mathcal{Q}$, two indicators are considered: the required robot count N_i and the successor count $N_{\text{suc},i}$. The normalized parameters $p_{i,1}$ and $p_{i,2}$ are computed as follows.

$$p_{i,1} = \frac{N_i}{\sum_{q_j \in \mathcal{Q}} N_j}, \qquad p_{i,2} = \frac{N_{\text{suc},i}}{\sum_{q_j \in \mathcal{Q}} N_{\text{suc},j}} \tag{4}$$

The entropy weights w_k are computed as follows.

$$e_k = -\frac{1}{\ln |\mathcal{Q}|} \sum_{q_i \in \mathcal{Q}} p_{i,k} \ln p_{i,k}, \qquad k \in \{1,2\} \tag{5}$$

$$w_k = \frac{1 - e_k}{\sum^2_{m=1}(1 - e_m)}, \qquad k \in \{1,2\} \tag{6}$$

where e_k denotes the normalized entropy of indicator k, $k \in \{1,2\}$ (with $k = 1$ for required robot count and $k = 2$ for successor count), and $|\mathcal{Q}|$ is the number of subtasks. In the equation of w_k, w_k is the entropy weight of indicator k, and m is the indicator index used in the normalization denominator so that $\sum^2_{k=1} w_k = 1$. The entropy-weighted importance of subtask q_i is computed as follows.

$$s_i = w_1 p_{i,1} + w_2 p_{i,2} \tag{7}$$

$$\bar{s}_i = \frac{s_i}{\sum_{q_j \in \mathcal{Q}} s_j} \tag{8}$$

where s_i is the unnormalized importance score of subtask q_i, and $\bar{s}_i$ is its normalized score. The normalization enforces $\sum_{q_i \in \mathcal{Q}} \bar{s}_i = 1$, so $\bar{s}_i$ can be used directly as a weight in center computation. Let x_i denote the geometric center of subtask region q_i. The free-market center is computed as follows.

$$x_{\text{free}} = \sum_{q_i \in \mathcal{Q}} \bar{s}_i x_i \tag{9}$$

where x_{free} is the center of the free market. This equation is a weighted average of subtask-region centers; larger $\bar{s}_i$ makes subtask q_i exert stronger influence on x_{free}. If x_{free} lies in a forbidden region or outside the workspace, the final free-market location is set to the nearest feasible point. In this work, a robot is considered to have arrived at the free market when it reaches a feasible point that is as close as possible to the free-market center and lies within the communication range of at least one robot already in the free market (if any).

4) Collaborative Subtask Coordination: For collaborative subtasks, robots adopt a leader-based coordination strategy, where one robot serves as a temporary leader to collect arrivals, synchronize decisions, and issue recruitment actions. Compared with fully leaderless coordination, this design reduces negotiation overhead and decision inconsistency, thereby improving coordination efficiency and execution robustness.

The handling flow is as follows: robots first synchronize arrivals under leader coordination; if the required team cannot be formed within the collaboration waiting time, timeout-triggered recruitment is activated; if recruitment still fails, the subtask is regarded as failed, and failed subtasks are reassigned in the free market.

Upon arriving at a subtask region, robot r_k broadcasts an arrival signal $e_k = \{t^k_e, q_k, a_k\}$, where t^k_e is the arrival

timestamp at the task region, and q_k and a_k denote the current subtask and the action performed by r_k, respectively. The robot that arrives at the task region first is selected as the leader r_l to prevent conflicts from asynchronous arrivals. If the number of robots that have arrived within T_{wait} seconds satisfies the requirement of subtask q_k, the leader broadcasts a start signal at that moment, and the robots start executing the collaborative subtask. Otherwise, if the waiting time exceeds T_{wait}, timeout-triggered recruitment is initiated.

Algorithm 1 Timeout-Triggered Recruitment

Input: subtask q_k, arrived robot set R_{arr}, robot-state set S_l, free-market location x_{free}, free-market robot set R_f, reachable robot set R_b
Output: status $\in$ SUCCESS, FAILURE
if $|R_{\text{arr}}| < N_t^k$ **then**
  $R_{\text{fit}} \leftarrow \{r_j \in\in R_b \mid \text{type}(r_j) \in \text{Req}(q_k)\}$
  if $|R_{\text{arr}} \cup R_{\text{fit}}| \geq N_t^k$ **then**
    sort R_{fit} by t_a^m in S_l and recruit first $(N_t^k - |R_{\text{arr}}|)$
    update T_{wait} via Eq.10
    if waiting time $t_{\text{wait}} < T_{\text{wait}}$ **then**
      return SUCCESS
    else
      return FAILURE
    end if
  end if
  $R_{\text{fit}}^f \leftarrow r_j \in R_f \mid \text{type}(r_j) \in \text{Req}(q_k)$
  if $|R_{\text{arr}} \cup R_{\text{loc}} \cup R_{\text{fit}}^f| < N_t^k$ **then**
    return FAILURE
  end if
  $r_p \leftarrow \arg\min_{r \in R_{\text{arr}}} d(v_p, x_{\text{free}})$
  dispatch r_p to free market
  compute timeout T_{leave}^p via Eq.11
  if recruitment success **then**
    return SUCCESS
  else if waiting time $t \geq T_{\text{leave}}^p$ **then**
    return FAILURE
  end if
end if

The collaboration waiting time T_{wait} is dynamically determined using leader r_l's robot-state information set $S_l(\tau_n)$. T_{wait} is derived from the earliest arrival time T_{early} and the latest arrival time T_{late}, as defined in Eq.(10).

$$T_{\text{wait}} = \left(1 + \beta_w \frac{N_t^k}{N_{\max}}\right)(T_{\text{early}} - T_{\text{late}}) \tag{10}$$

where β_w is an adjustment coefficient, β_w is set to 1.25 based on experiments. q_k is the subtask currently executed by leader r_l, N_t^k is the number of robots required for subtask q_k, and $N_{\max}$ is the maximum robot requirement across all subtasks. This design uses the arrival-time spread $(T_{\text{early}} - T_{\text{late}})$ as the baseline waiting window and scales it with $N_t^k/N_{\max}$, so subtasks requiring more robots are given a proportionally longer waiting time to improve collaboration success while avoiding excessive delay for smaller teams. The rationale is that larger teams are harder to synchronize under limited communication: more robots imply more dispersed initial positions and greater arrival-time variability, so the probability that all required robots arrive within a short window is lower.

When leader r_l observes that the elapsed waiting time has reached T_{wait} seconds and the arrived team size is still below the required cardinality for q_k, it first performs local recovery. Specifically, it scans robots that are reachable within its communication neighborhood, either directly or through multi-hop relays, for available robots (robots that are neither executing another subtask nor already committed to recruitment) and checks whether each robot can fill a vacancy of q_k, i.e., whether its type matches one of the still-missing role requirements of q_k. It then counts how many missing roles can be filled immediately. If the local candidates are sufficient, leader r_l recruits them in ascending order of estimated travel time to the subtask region of q_k and updates T_{wait} from their predicted arrival times, so that the recruited robots are expected to arrive within the updated waiting window and can still join the current collaborative execution of q_k. If the updated T_{wait} is still exceeded, leader broadcasts task-failure message for q_k.

If the local candidates are insufficient, leader r_l expands the search to the free market and builds a backup candidate pool from robots explicitly listed in the free-market ID set $\mathcal{R}_f$ that can fill the remaining vacancies of q_k (i.e., robots whose capabilities match the missing role requirements).

The leader then checks whether the union of the local candidates and recruitable free-market robots is sufficient to fill all remaining roles of q_k. If the union is insufficient, subtask q_k is then treated as a failed subtask. If the union is sufficient, leader r_l selects a recruiter r_p with the shortest travel time to the free market and starts recruitment. Recruiter r_p follows a first-come-first-served strategy among received bids. Upon receiving a bid from robot r_c, recruiter r_p sends a lock request $\text{I}_{\text{lock}}^{\text{p}} = \{q_p, \text{ID}_c\}$ to r_c, where q_p is the recruited subtask and ID_c is r_c's ID. The uncommitted bidder r_c replies with $\text{I}_{\text{agree}}^{\text{c}} = \{q_p, \text{ID}_c\}$ and becomes locked. Once the recruitment completes, r_p broadcasts $\text{I}_{\text{finish}}^{\text{p}} = \{q_p\}$, and the locked robots then proceed to q_p. Robot r_p broadcasts recruitment messages until the required robots are obtained or its stay in the free market exceeds T_{bid} seconds. Leader r_l declares subtask failure if recruitment is unsuccessful within T_{leave}^p seconds.

$$T_{\text{leave}}^p = \left(1 + \zeta \frac{d_{\text{leave}}}{d_{\max}}\right)\left(\frac{2d_{\text{leave}}}{v_p}\right) + T_{\text{bid}} \tag{11}$$

where $d_{\max}$ is the maximum inter-target region distance, and ζ is the adjustment coefficient. Based on experiments, ζ is set to 0.75 in this paper. d_{leave} denotes the distance from recruiter r_p to the free market. The first term in Eq. (11) estimates the round-trip travel time between the subtask region and the free market, while T_{bid} reserves time for local bidding and message dissemination in the market. In this paper, T_{bid} is computed as

$$T_{\text{bid}} = |\mathcal{R}| \frac{1}{f_{\text{com}}} N_{\text{round}} + t_{\text{comp}}, \tag{12}$$

where $|\mathcal{R}|$ is the total number of robots, N_{round} is the number of recruiter–bidder message rounds required by the recruitment process, and t_{comp} is the computation time. t_{comp} is set to 2.0 seconds in this paper. We assume that once a message

is sent, it is received instantaneously by all robots within communication range. The scaling factor $(1+\zeta d_{\text{leave}}/d_{\max})$ makes the timeout adaptive to travel distance, so leaders avoid declaring failure too early for distant recruiters and avoid excessive waiting for nearby recruiters.

If recruitment is still unsuccessful within T^p_{leave} seconds, subtask q_k is declared failed, and the leader notifies all robots currently executing q_k. Failed subtasks are appended to each robot's failed-subtask list and are temporarily skipped until robots enter the free market, where these subtasks are re-evaluated and reassigned (Sec. IV-C3).

This design separates immediate collaboration recovery from system-level reassignment for two reasons. First, deferring reassignment to the free market avoids frequent route switching under sparse communication, which reduces oscillatory replanning caused by inconsistent local views. Second, by concentrating failed-subtask redistribution in a connected information-exchange region, robots can make reassignment decisions with more complete team-state information, improving the success rate of re-execution and reducing cascading failures.

D. Task Failure Recovery

Distributed approaches may encounter task failures under limited communication. In this subsection, task-failure recovery is divided into two parts: i) failure handling for non-collaborative subtasks, and ii) failure handling for collaborative subtasks.

1) Failure Handling for Non-collaborative Subtasks: For non-collaborative subtasks, failure specifically means that the assigned robot becomes unavailable due to faults or other unexpected events and therefore cannot complete the subtask. To address this issue, each robot uses a timeout-based rule to assess peer status. For a non-collaborative subtask currently executed by robot r_m, if no completion message for that subtask is received within a specified threshold, other robots infer that r_m may have failed, which may render the subtask uncompletable.

Specifically, if robot r_m's timestamp in the robot-state information set $S_k(\tau_n)$ lags the current time τ_{now} by more than $T^{k,m}_{\text{late}}$, robot r_k treats it as outdated and reassigns q_m if r_k can perform a_m, q_m is non-collaborative, and $q_m \neq q_k$. This reassignment rule is restricted to non-collaborative subtasks because collaborative subtasks are handled by a separate coordination mechanism, which avoids mixing different recovery logics.

Even for non-collaborative subtasks, delayed and incomplete information propagation may cause the same subtask to be assigned to multiple robots. Let $\mathcal{H}_k(q_m)$ denote the set of robots in $S_k(\tau_n)$ that are assigned to q_m. For each $r_h \in \mathcal{H}_k(q_m)$, robot r_k computes a timeout candidate $T^{k,h}_{\text{late}}$ by Eq. 14 and uses the largest one as the timeout for q_m:

$$T^{k,m}_{\text{late}} = \max_{r_h \in \mathcal{H}_k(q_m)} T^{k,h}_{\text{late}}. \tag{13}$$

$$T^{k,h}_{\text{late}} = \alpha_{\text{late}} \left(1 + \left\lceil \frac{d_{k,h}}{D} \right\rceil\right) \max\left(T_{\text{enc}}, \frac{1}{f_{\text{com}}}\right) + t^h_s + \max_{a_i \in \text{LA}(q_m)} \text{LC}(a_i) \tag{14}$$

$$T_{\text{enc}} = \frac{1}{4Dv_a\rho_r} \tag{15}$$

where α_{late} is a tuning parameter. In this paper, α_{late} is set to 2.5 based on experimental results. The value is chosen to be greater than 1 so that the practical information expiration time remains above its theoretical estimate. $d_{k,h}$ denotes the distance between robot r_k's current position and robot r_h's estimated position. The estimated position of r_h is estimated based on its last known position p_h at the time of the most recent communication, together with its task plan. The estimation is carried out as follows: the current time is compared with the subtask start time in the plan to determine which task the robot is executing. If the current time exceeds the planned completion time of the task, the robot is considered to be in the free market. Otherwise, the robot is assumed to move at a constant speed along a straight line between task points, and its current position is estimated accordingly. T_{enc} is the average encounter time [36] , where v_a is the average speed of robots and ρ_r is the robot density.

T_{enc} denotes the average encounter time [36], where v_a is the average speed of the robots and ρ_r is the robot density.

2) Failure Handling for Collaborative Subtasks: Failed collaborative subtasks are handled through free-market-based reassignment.To be noticed, the communication network among robots in the free market is connected. Once a robot enters the free market, failed subtasks are reallocated through a recruiter–bidder mechanism after information is synchronized, enabling robots to recover from collaboration failures using updated information.

When a robot finds that its current optional subtask set contains only failed subtasks, it enters the free market for reassignment. After arriving at the free market, robot r_k first synchronizes recruitment messages and state information, and then checks whether it can serve as a recruiter. Robot r_k becomes a recruiter only when it receives no valid recruitment message for any compatible subtask. If it becomes a recruiter, it then selects one failed subtask from l^k_{fail} according to the shortfall-and-distance rule and publishes a recruitment message for that subtask.

For each required robot type $j \in \mathcal{R}_{\text{type}}$, define

$$\mathcal{R}_{\text{rem}}(t,j) = \mathcal{R}_{\text{free}}(t,j) - \sum_{\hat{r} \in \mathcal{R}^{\text{earlier}}_k} \mathcal{R}_{\text{req}}(q_{\hat{r}}, j), \tag{16}$$

where $\mathcal{R}_{\text{free}}(t,j)$ is the number of free-market robots of type j at time t, $\mathcal{R}_{\text{req}}(m,j)$ is the required number of type-j robots for subtask q_m, and $\mathcal{R}^{\text{earlier}}_k \subseteq \mathcal{R}^{\text{act}}_{\text{rec}}$ is the set of recruiters that published earlier than r_k.

For each subtask $q_m \in l^k_{\text{fail}}$, the shortfall is calculated as follows.

$$\Delta(q_m) = \sum_{j \in \mathcal{R}_{\text{type}}} \max(0,\ \mathcal{R}_{\text{req}}(q,j) - \mathcal{R}_{\text{rem}}(t,j)). \tag{17}$$

Recruiter r_k selects the subtask with the minimum shortfall.

$$q_k^* = \arg \min_{q_m \in l_{\text{fail}}^k} \Delta(q_m). \tag{18}$$

If multiple candidates share the same minimum shortfall, r_k chooses the nearest subtask region. After determining the subtask to be handled first, r_k publishes recruitment message $\mathrm{I}_{\text{bid}}^k = \{\tau_k, r_k, q_k^*, s_k\}$, where τ_k is the publishing time and s_k specifies required robot types and quantities.

In the free market, any robot that is not acting as a recruiter is treated as a bidder. Bidder r_c applies according to task priority and recruitment-message publishing time. Let $\mathcal{M}_c(\tau_n)$ be the set of active recruitment messages received by bidder r_c at time τ_n, and let

$$\mathcal{M}_c^{\text{com}}(\tau_n) = \{\mathrm{I}_{\text{bid}}^p \in \mathcal{M}_c(\tau_n) \mid r_c \text{ is compatible with } q_p\}. \tag{19}$$

For each message I_{bid}, define $\rho(q_p)$ as the priority rank of subtask q_p (smaller value means higher priority). The bidder choice is

$$p_c^* = \arg \min_{p:\, \mathrm{I}_{\text{bid}}^p \in \mathcal{M}_c^{\text{com}}(\tau_n)} \big(\rho(q_p), \tau_p\big), \tag{20}$$

where minimization is lexicographic: bidders first select the highest-priority subtask, and if priorities are equal, they select the earliest published recruitment message. In this paper, high-priority subtasks refer to emergency subtasks issued by leader-dispatched recruiters, whereas low-priority subtasks refer to regular subtasks that are failed or currently unassigned.

During recruitment, if a recruiter r_p receives a compatible recruitment message $\mathrm{I}_{\text{bid}}^h$ that it can execute as a bidder, and the currently available free-market robots are insufficient to satisfy both recruitments simultaneously, r_p compares $(\rho(q_h), \tau_h)$ with $(\rho(q_p), \tau_p)$. If $\rho(q_h) < \rho(q_p)$, or if $\rho(q_h) = \rho(q_p)$ and $\tau_h < \tau_p$, then r_p relinquishes its recruiter role, cancels its own recruitment, and switches to bidder mode for $\mathrm{I}_{\text{bid}}^h$.

Recruiter r_p follows a first-come-first-served strategy among received bids the same as Sec. IV-C4. Upon receiving a bid from robot r_c, recruiter r_p sends a lock request $\mathrm{I}_{\text{lock}}^{\text{p}} = \{q_p, \mathrm{ID}_c\}$ to r_c, where q_p is the recruited subtask and ID_c is r_c's ID. The uncommitted bidder r_c replies with $\mathrm{I}_{\text{agree}}^{\text{c}} = \{q_p, \mathrm{ID}_c\}$ and becomes locked. Once the recruitment completes, r_p broadcasts $\mathrm{I}_{\text{finish}}^{\text{p}} = \{q_p\}$, and the locked robots then proceed to q_p. Robots are unlocked if they receive no messages from r_p within T_{send} seconds (presumed recruiter failure) or if higher-priority subtasks arrive. Here, T_{send} is set as

$$T_{\text{send}} = \left(1 + \left\lceil \frac{d_{c,p}}{D} \right\rceil\right) \frac{1}{f_{\text{com}}}, \tag{21}$$

where $d_{c,p}$ is the distance from bidder r_c to recruiter r_p, D is the communication range, and f_{com} is the communication frequency. They then transmit $\mathrm{I}_{\text{unlock}}^{\text{c}} = \{q_p, \mathrm{ID}_c\}$ to prompt r_p to remove them from the locked robot list l_{lock}.

E. Patrol Mechanism for Task Blocking

Temporal logic constraints can cause blocking while robots wait for prerequisite subtasks to be confirmed. For example, if subtask "inspect Zone B" is constrained to start only after "verify tunnel entrance" is completed, a robot assigned to Zone B may remain idle when the completion message from the entrance team cannot be received in time under limited communication. To address this issue, a patrol mechanism is proposed to proactively gather and relay prerequisite status information, thereby releasing blocked subtasks earlier. For each subtask $q_i \in Q_k$ executable by robot r_k, let $Q_{\text{pre}}^{i,k}$ denote its prerequisite subtasks. The prerequisites set is $Q_{\text{pre}}^k = \bigcup_{q_i \in Q_k} Q_{\text{pre}}^{i,k}$. r_k maintains a prerequisite timetable $\mathrm{Table}_{\text{pre}}^{\text{k}}$ with estimated completion time t_{pre}^j for all $q_j \in Q_{\text{pre}}^k$. t_{pre}^j is computed using the same form as $T_{\text{late}}^{k,m}$ in Sec.. For subtasks constrained only by "do not start before the prerequisite starts," the same estimation logic is used, except that the prerequisite start time is adopted, which is obtained by subtracting the prerequisite execution duration from its estimated completion time. For subtasks constrained only by "do not start before the prerequisite starts," the same estimation logic is used, except that the prerequisite start time is adopted, which is obtained by subtracting the prerequisite execution duration from its estimated completion time.

A robot triggers the patrol mechanism only when two conditions are simultaneously satisfied: i) there is no currently unassigned optional subtask that can be executed immediately, and ii) for at least one pending subtask q_i, the current time exceeds the latest estimated completion time of its prerequisites, i.e., $t_{\text{now}} > \max_{q_j \in Q_{\text{pre}}^{i,k}} t_{\text{pre}}^j$. This means the robot has waited beyond the expected prerequisite completion horizon but still cannot confirm prerequisite satisfaction from local information. In this case, the robot starts patrol to actively collect missing status updates. The initial patrol targets include all regions associated with unresolved prerequisite subtasks and the free market. During patrol, the target set is refined as follows: i) remove regions whose prerequisites are already completed or reliably verified, ii) skip prerequisites whose successor subtasks have already started, iii) omit prerequisites whose own predecessor conditions are still unmet, and iv) update optional-subtask candidates and their estimated completion times using newly gathered information. Once at least one executable optional subtask is identified, the robot exits patrol and switches to task execution.

After patrol, if the estimated completion time is updated, robots will re-patrol based on new timings; otherwise, robots will wait t_p seconds before next patrol.

$$t_p = \max_{q_j \in Q_{\text{pre}}^k} \left(\frac{d_f^j}{v_a} + \max_{a_i \in \mathrm{LA}(q_j)} \mathrm{LC}(a_i) \right) \tag{22}$$

where d_f^j is the free-market-to-subtask distance. t_p assumes the subtask has failed and will be reassigned in the free market.

V. Algorithm Analysis

This section proves that, under the assumptions in Sec.III-B and a finite subtask set (i.e., $|Q| < \infty$), the proposed algorithm guarantees task completion.

Theorem 2: Completion of unfinished regular subtasks allocated via the free market is guaranteed.

Proof: Consider an unfinished subtask q_1 requiring robot types $\mathcal{R}_{\text{type}}$, with required quantities $\mathcal{R}_{\text{req}}(q_1, j)$ for type $j \in \mathcal{R}_{\text{type}}$. Let $\mathcal{R}_{\text{free}}(t', j)$ denote the number of robots of type j in the free market at time t', and take any $r_k \in \mathcal{R}_j$. We first show r_k enters the free market within finite time T^k_{bound}. For q_1, r_k falls into one of three cases: i) $q_1 \in l^k_{\text{fail}}$; ii) insufficient teammates; iii) q_1 has been assigned to other robots. Case iii) becomes ii) after time T_{late} if q_1 is a non-collaborative subtask, which is bounded by $T^{\max}_{\text{timeout}} \leq \alpha_{\text{late}} \left(1 + \left\lceil \frac{d_{\max}}{D} \right\rceil\right) \max\left(T_{\text{enc}}, \frac{1}{f_{\text{com}}}\right) + t^{\max}_s + L^{\max}_{\text{task}}$ according to Eq. (14) and Eq. (13), where $d_{\max}$ is the maximum workspace distance, $t^{\max}_s$ is the latest estimated subtask start time in $S_k(\tau_n)$, and $L^{\max}_{\text{task}} = \max_{q_j \in \mathcal{Q}} \max_{a_i \in \text{LA}(q_j)} \text{LC}(a_i)$. Assume that the maximum execution time of a subtask for r_k is $T^{\max,k}_{\text{task}}$. The subtask execution time is finite. With co-safe LTL, $|\mathcal{Q}| < \infty$. Thus, robot r_k will enter the free market within a finite time $T^k_{\text{bound}} \leq |\mathcal{Q}|\left(T^{\max,k}_{\text{task}} + T^{\max}_{\text{timeout}}\right)$.

Now consider three cases for q_1: i) Recruitment proceeds without interruption by high-priotity subtasks. According to assumption iii) and the condition that robots do not issue recruitment requests that exhaust free-market resources, all robots capable of q_1 will enter the free market within a finite time. Therefore, there exists a time $t' \in (t_0, \infty)$ such that $\forall j \in \mathcal{R}_{\text{type}}, \quad |\mathcal{R}_{\text{free}}(t', j)| \geq |\mathcal{R}_{\text{req}}(q_1, j)|$. So q_1 can be successfully allocated. ii) Recruitment process is interrupted by emergency subtask recruitment. Since $|\mathcal{Q}| < \infty$, and any departing robot r_k is guaranteed to rejoin the free market within T^k_{bound}, interruptions terminate in finite steps, reducing to case i); iii) If q_1 remains unposted, it is because all capable robots are already engaged in earlier recruitments. Since every existing recruitment falls into case i) or ii), q_1 will eventually be posted. Hence, in all cases, q_1 will eventually be posted and successfully allocated.

Theorem 3: The proposed algorithm guarantees all subtasks will be completed under assumptions ii) and iii).

Proof: The feasibility of CBTA is discussed in [5]. Under assumption iii), incomplete subtasks arise from: i) missing prerequisite subtask information, ii) insufficient teammates, or iii) robots expected to participate in a collaborative subtask do not reach the target region within the collaboration waiting time or the subsequent recruitment timeout. Case i) is resolved by the patrol mechanism. In cases ii) and iii), insufficient teammates and subtask failures eventually lead robots to enter the free market, since $|\mathcal{Q}| < \infty$. By Lemma 1, all subtasks will ultimately be completed.

Theorem 4: Consider a discrete workspace W, a team of n robots of m types, and a valid specification $\varphi \in \text{LTL}$. Assume that there exists a feasible robot path satisfying φ. Then, the proposed algorithm is guaranteed to find a robot path that satisfies φ.

Proof: The key idea in the proof is to first show that feasible words are preserved after automaton pruning and then use this fact to establish the existence of feasible paths, task allocations, and executable robot trajectories. The detailed proof follows the sequence of feasibility arguments for the DFS-based path search, the task decomposition, CBTA task allocation, and path planning for each assigned subtask.

Specifically, the automaton pruning step removes only infeasible transitions, and by Lemma 1, any word accepted by the original NBA A_φ has a corresponding word accepted by the pruned automaton A^-_φ. Therefore, feasible words are preserved. Next, since DFS is complete over finite graphs, any accepting path in A^-_φ will be discovered. The accepting path is then decomposed into a sequence of subtasks, which by construction respects the temporal ordering encoded in φ. Given the existence of a feasible path that satisfies all subtasks and enough time, the CBTA-based allocation procedure is guaranteed to find a feasible assignment. If a subtask execution fails, Theorems 2 and 3 ensure that the subtask can be reassigned and successfully executed by another robot, guaranteeing completion of the overall plan. During execution, feasible path planning for each assigned subtask exists, as all transitions in $\mathcal{B}^-$ correspond to realizable motions in the workspace.

VI. RESULT

The algorithm is evaluated through simulations and physical experiments. LTL formulas are converted to NBA using LTL2BA [37]. Simulations run in Python 3 on an 11th Gen Intel® Core™ i7-11390H with 16 GB RAM. Physical experiments use Ubuntu 18.04 with ROS.

A. Numerical Simulation

We evaluate the proposed framework under communication-constrained settings, with a focus on its effectiveness in handling task blocking, dynamic robot changes, collaborative coordination, and failure recovery. To this end, the proposed D^2TAPR is examined through four test scenarios:

i) Test 1 evaluates the patrol mechanism through an ablation study. For each instance, the patrol mechanism is enabled or disabled under the same task setting, while the communication range is varied to examine how efficiently blocked prerequisite subtasks are resolved and how this affects the final task completion rate.

ii) Test 2 compares timeout-triggered recruitment with with timeout-based local recruitment [32] under multiple communication ranges. This comparison is used to evaluate the impact of our timeout-triggered recruitment on collaboration success and overall completion efficiency.

iii) Test 3 evaluates free-market-based reassignment in preventing repeated subtask failures, using failure-to-tail reassignment as a baseline with $D = 3$. The goal is to examine whether free-market-based reassignment can reduce repeated failure chains and improve recovery stability under limited communication.

iv) Test 4 examines D^2TAPR's adaptability to dynamic robot changes (e.g., robots joining, or temporarily becoming unavailable during execution). We evaluate whether the framework can maintain feasible allocation updates and continue task progress without global replanning.

The experimental setup consists of 12 target regions, denoted by $l = \{l_1, l_2, \ldots, l_{12}\}$, and a 12-robot team comprising three heterogeneous robot types: type1, type2, and type3 . The spatial arrangement of regions and robot deployment

follows Fig. 4. To evaluate robustness under different task-location assignments, we treat the region labels $l_1 \sim l_{12}$ as permutable and generate 25 randomized benchmark instances (Inst.1 $\sim$ Inst.25) by varying the label-to-location mapping while keeping the underlying workspace geometry unchanged.

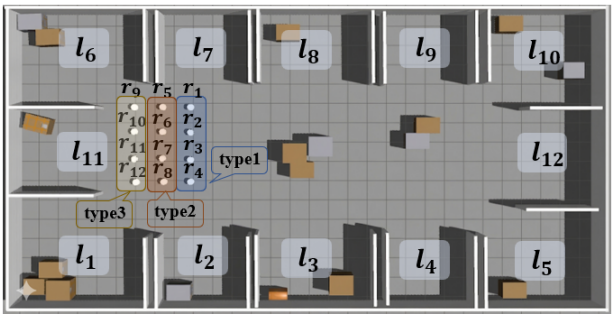

Fig. 4. Simulation environment for Tests. The 30×15 simulation environment contains 12 target regions $l_1 \sim l_{12}$ and 12 robots $r_1 \sim r_{12}$.

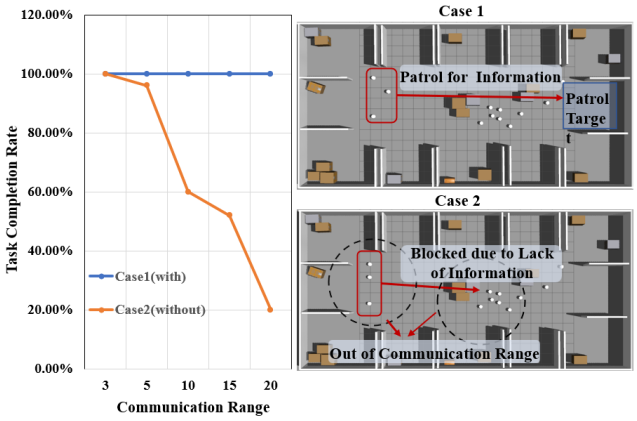

Fig. 5. The results of Test 1. Left: task completion rates under different communication ranges. Right: simulation snapshots and qualitative explanation of the behavioral differences between Case 1 (with patrol) and Case 2 (without patrol).

1) Test 1: Consider a LTL-specified global task $\varphi = \Diamond((l^{12}_{2,1} \wedge l^{12}_{1,1}) \wedge \Diamond((l^{7}_{2,2} \wedge \neg(l^{12}_{2,1} \wedge l^{12}_{1,1})) \wedge \Diamond(l^{6}_{2,1} \wedge \neg l^{7}_{2,2}))) \wedge \Diamond l^{2}_{3,1} \wedge \Diamond l^{3}_{3,1} \wedge \Diamond(l^{9}_{3,3} \wedge \neg p_4)$. After preprocessing φ, the resulting subtask set is $\Omega_\varphi = \{q_1, q_2, q_3, q_4, q_5, q_6\}$, where $q_1 = \{1, \{l^{9}_{3,3}\}, \{p_4\}\}$, $q_2 = \{2, \{l^{3}_{3,1}\}, \{\}\}$, $q_3 = \{3, \{l^{2}_{3,1}\}, \{\}\}$, $q_4 = \{4, \{l^{12}_{2,1}, l^{12}_{1,1}\}, \{\}\}$, $q_5 = \{5, \{l^{7}_{2,2}\}, \{\}\}$,$q_6 = \{6, \{l^{6}_{2,1}\}, \{\}\}$. The objective of this experiment is to verify the causal role of the patrol mechanism in resolving task blocking under limited communication. To this end, we evaluate Inst.1$\sim$Inst.25 under two settings: Case 1 (with patrol) and Case 2 (without patrol), while keeping task instances, robot settings, and communication ranges identical. The only algorithmic difference is whether robots are allowed to proactively get information.

As shown in Fig. 5, when communication range decreases, Case 2 exhibits a significant decline in task completion rate because blocked robots cannot reliably obtain prerequisite updates and therefore remain in waiting states for long periods. Notably, Case 2 can still complete some instances at small communication ranges when a blocked robot happens to encounter another robot that already carries the required prerequisite information; however, this information-recovery path is opportunistic and unstable, and its success strongly depends on workspace geometry, robot trajectories, and task-region distribution. In contrast, Case 1 maintains a 100% completion rate across all tested ranges: patrol converts passive waiting into active information acquisition, closes prerequisite-information gaps in a structured way, and enables blocked subtasks to be released in time without relying on chance encounters. These results highlight the key advantage of the patrol mechanism: it provides predictable and scenario-robust information recovery under communication constraints, rather than probabilistic recovery determined by incidental robot meetings.

2) Test 2: Considering the same global task ϕ as in Test 1, we further evaluate timeout-triggered recruitment. Timeout-triggered recruitment (TTR) is compared with timeout-based local recruitment (TLR) [32]. The two methods differ only in how replacement candidates are searched after timeout: TLR selects from robots currently reachable by the leader (usually the nearest one), while TTR can dispatch a recruiter to the free market to obtain additional qualified robots outside the local communication neighborhood. In each run, one robot is randomly removed during execution of q_1 to create the same failure trigger for both methods. Performance is measured by the completion time of type-3-related subtasks (T3-subtask time) under different communication ranges. We use this metric instead of total mission time because it directly captures recovery efficiency for the affected collaborative chain, while total mission time can be perturbed by unrelated subtasks.

Fig. 6 and Table I show that TTR yields shorter T3-subtask time in most instances, with the largest gains at small communication ranges. The reason is that, under sparse connectivity, TLR is constrained by local visibility and may not find a feasible replacement quickly, whereas TTR expands the search scope through the free market and therefore restores collaboration earlier. When communication range is large, the free market is effectively inside the connected neighborhood, so TTR and TLR have similar candidate sets and similar performance. The exceptions (e.g., Inst.2 and Inst.6) are also explainable: both methods behave similarly because q_2 and q_3 are far from initial robot positions and no suitable type-3 robot is available in the free market at timeout; here the dominant bottleneck is temporary resource scarcity, not recruitment strategy. Overall, the experiment indicates that TTR improves robustness mainly by enlarging the effective replacement-search region under communication constraints.

3) Test 3: In scenarios with multiple collaborative subtasks under limited communication, repeated failures can propagate across temporally dependent tasks and substantially delay mission completion. This test evaluates whether free-market-based reassignment (FMR) can reduce such cascading delays compared with failure-to-tail reassignment (F2TR), where each failed subtask is appended to the end of the local plan and retried later. The comparison is conducted over Inst.1$\sim$Inst.25 under $D = 1.5m$using task completion time as the primary metric.Consider the LTL-specified task $\varphi_1 = \Diamond l^{11}_{1,2} \wedge \Diamond l^{7}_{1,2} \wedge \Diamond l^{2}_{1,2} \wedge \Diamond((l^{2}_{2,1} \wedge \neg l^{9}_{2,2}) \wedge \Diamond l^{9}_{2,2})$. Let $\Omega_\varphi = \{q_1, q_2, q_3, q_4, q_5\}$,

TABLE I
T3-SUBTASK TIME GAP BETWEEN TTR AND TLR UNDER DIFFERENT COMMUNICATION RANGES. ALL TIME GAPS ARE IN SECONDS (S).

Communication Range (m)	Inst. 1	Inst. 2	Inst. 3	Inst. 4	Inst. 5	Inst. 6	Inst. 7	Inst. 8	Inst. 9	Inst. 10	Inst. 11	Inst. 12	Inst. 13
3	11.29	0.03	19.56	27.82	16.75	0.12	16.22	52.07	21.69	21.69	0.32	17.09	36.36
5	16.26	0.12	27.73	37.31	24.61	0.23	15.48	10.20	15.49	15.49	0.12	0.32	0.32
10	26.13	0.02	0.00	43.70	0.11	0.09	0.00	10.28	27.52	27.52	0.22	-0.25	0.32

Table I (continued)

Communication Range (m)	Inst. 14	Inst. 15	Inst. 16	Inst. 17	Inst. 18	Inst. 19	Inst. 20	Inst. 21	Inst. 22	Inst. 23	Inst. 24	Inst. 25
3	20.43	-0.19	23.10	18.41	13.53	0.32	19.91	26.17	0.27	16.93	22.85	0.12
5	16.57	-0.28	17.49	28.18	14.14	0.09	16.11	16.03	-0.13	27.78	15.56	0.09
10	23.12	0.32	6.74	0.18	-0.24	0.27	24.31	25.08	-0.25	-0.02	26.04	0.27

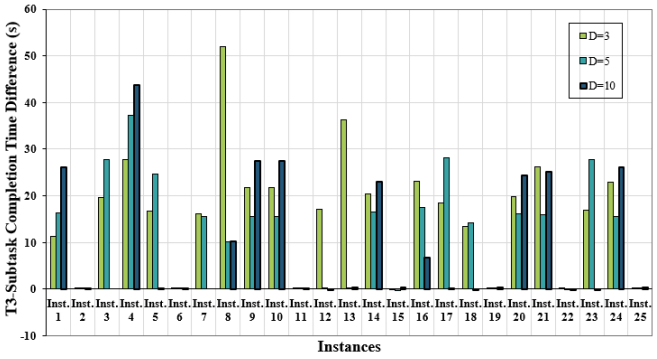

Fig. 6. Comparison between TTR(proposed) and TLR. The positive values indicating superior performance of TTR.

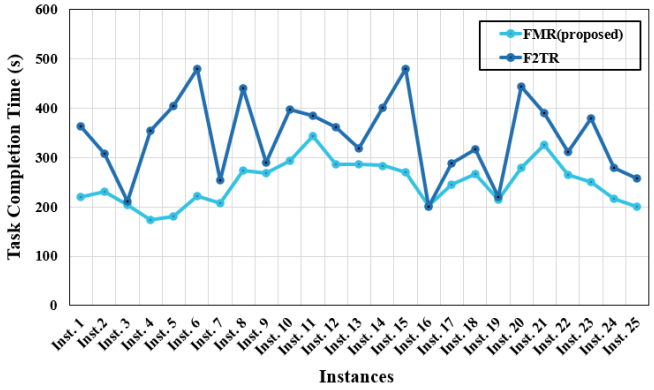

Fig. 7. Comparison between FMR (proposed) and F2TR.

where $q_1 = \{1, \{l^2_{2,1}\}\}$, $q_2 = \{2, \{l^9_{2,2}\}\}$, $q_3 = \{3, \{l^2_{1,2}\}\}$, $q_4 = \{4, \{l^7_{1,2}\}\}$, and $q_5 = \{5, \{l^{11}_{1,2}\}\}$. Five robots are deployed: r_1, r_2, r_3 (type 1) and r_4, r_5 (type 2).

To trigger representative collaborative failures, one type-1 robot is removed before the second (in execution order) type-1 collaborative subtask is completed. This setting is critical because the team has minimal redundancy: a single removal can break feasibility of the current collaboration and force reallocation. Under F2TR, this often leads to deferral and repeated waiting in downstream subtasks. In contrast, FMR synchronizes failure information in the free market and reallocates with a wider, updated candidate set, which is expected to reduce repeated failures and shorten recovery latency. From a mechanism perspective, the key difference is whether recovery decisions are made with fragmented local views (F2TR) or with synchronized team-state information (FMR).

As shown in Fig 7, FMR generally outperforms F2TR across most instances, indicating that the benefit is systematic rather than case-specific. As shown in Fig 8, the key reason is that, under dense collaborative dependencies and limited communication, a single failure often creates a "failure–delay–reassignment" loop: delayed recovery of one subtask postpones its successors, which increases the probability of additional waiting and repeated failures. F2TR tends to amplify this loop because failed subtasks are deferred to the tail and revisited with fragmented local information. In contrast, FMR interrupts the loop by synchronizing failure states in the free market and reallocating from a broader, updated candidate set, which shortens recovery latency and reduces repeated failure cycles.

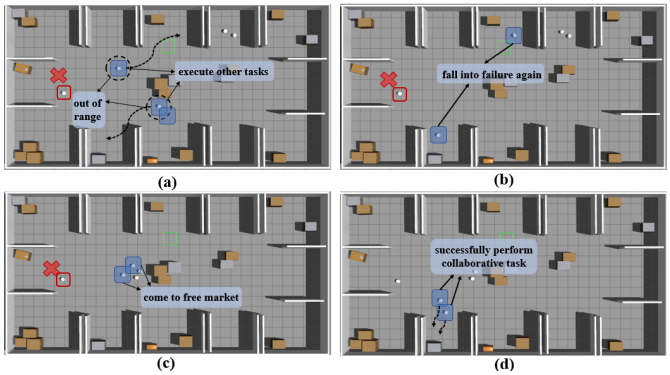

Fig. 8. Schematic comparison of collaborative-failure handling processes in F2TR and the proposed method. Subfigures (a) and (b) illustrate the F2TR process, while (c) and (d) illustrate the process of the proposed method.

The smaller gaps in instances such as Inst.3 and Inst.16 are also informative: when spatial layout and timing naturally permit sufficient information exchange, even local-tail recovery can approach the performance of FMR. This suggests that FMR's main advantage appears in communication-stressed or coordination-dense cases, i.e., precisely the operating conditions where robustness is most needed. Overall, the results support that FMR improves not only average completion efficiency but also resilience to chained collaborative-subtask disruptions.

4) Test 4: At $D = 10\,\text{m}$, with regions $l_1 \sim l_{12}$ arranged as shown in Fig. 4, Test 2 evaluates the proposed framework under dynamic changes in team composition by introducing both robot addition and robot removal during task execution.

At $t = 15$, a new type-3 robot, r_{13}, joins the system. After entering the communication range and synchronizing with the team, r_{13} is incorporated into the candidate set and assigned to subtask q_2. This online reallocation updates the task assignment according to the new team composition, thereby reducing the burden on the original team and yielding a plan suited to the expanded set of available robots.

Later, at $t = 80$, robot r_{10} is removed, leaving collaborative subtask q_1 understaffed. In response, the leader robot r_{11} keeps the remaining participants in a waiting state while checking whether the missing role can be filled locally. Once the waiting time exceeds T_{wait}, r_{11} activates the timeout-based recovery mechanism and dispatches r_9 as a recruiter to seek a qualified replacement. After recruitment succeeds, the team is re-formed, and q_1 is eventually completed by r_{11}, r_{13}, and r_9 (Fig. 9).

From a process perspective, this scenario captures the full evolution from disturbance to response and eventual recovery. Specifically, the removal of r_{10} first introduces the risk of collaborative-subtask failure due to an insufficient number of participants for q_1; the timeout-triggered recruitment mechanism then restores the required team size; and finally, task execution returns to a consistent and stable state. Therefore, the trajectory shown in Fig. 9 demonstrates not only successful task completion, but also the effectiveness of the proposed framework in supporting both collaborative-failure recovery and online task reallocation under dynamic team changes.

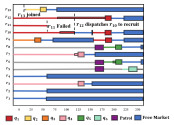

Fig. 9. Task timelines in Test 2. Wide blocks denote subtasks, narrow colored blocks indicate waiting. Gray lines mean no subtask and not in the free market, and other colored lines represent traveling toward the corresponding target region.

B. Physical Experiment

Consider the environment depicted in Fig. 10(a) where l_1 denotes the sorting area, l_2 the storage center, and l_3 the monitoring room. Four robots of three distinct types operate with a 2.0-meter communication range. The mission specification $\varphi = \mathcal{F}(l^1_{1,1} \wedge l^1_{3,1} \wedge l^1_{2,1}) \wedge \mathcal{F} l^2_{3,1} \wedge \mathcal{F} l^3_{1,1} \wedge \mathcal{F} l^2_{2,1} \wedge (\neg l^2_{2,1} \mathcal{U}\, l^4_{1,1}) \wedge (\neg l^3_{1,1} \mathcal{U}\, (l^1_{1,1} \wedge l^1_{3,1} \wedge l^1_{2,1}))$ requires one robot of each type (1, 2, 3) to inspect, classify, and process goods in the sorting area l_3; a type-3 robot to inspect the storage area l_2; and upon completion of sorting tasks, a type-1 robot to report in the monitoring room l_3, after which a type-2 robot processes goods in storage area l_2.

Experimental results (Fig. 10) show that the proposed mechanisms are effective in a real communication-limited setting. First, the collaborative operation in l_1 is completed before the dependent actions in l_3, which is consistent with the temporal-order constraints in φ and indicates that prerequisite information is propagated in time to avoid logical blocking. Second, after leaving l_1, robot r_1 proceeds to l_3 while r_2 is temporarily outside direct 2.0-meter communication range; nevertheless, team coordination is preserved through free-market-mediated information exchange, so execution does not stall due to link interruption. Third, once the l_3-related subtask is confirmed complete, r_2 transitions to l_2 to execute the subsequent dependent task, showing that task handoff remains consistent even across intermittent connectivity.

Overall, the physical experiment validates three key properties in combination: i) correctness of temporally constrained execution, ii) communication resilience via the free market under range limitations, and iii) continuity of distributed task progression without centralized global replanning.

VII. CONCLUSIONS

In this letter, a distributed task allocation and planning method is developed for a heterogeneous multi-agent system subject to communication constraints and linear temporal logic specifications. Each agent can obtain its own plan by the received or predicted task information. Simulation results demonstrate the effectiveness of the proposed method.

REFERENCES

[1] Y. Qu, B. Xin, Q. Wang, R. Li, and Z. Du, "A novel memetic algorithm for distributed shape formation of swarm robots with both acceleration and velocity constraints," *Science China Information Sciences*, vol. 67, no. 12, p. 222201, 2024.

[2] H. Fang, S. Wei, Y. Shi, and J. Ou, "Multi-agent ergodic surveillance under unknown target distribution," *Unmanned Systems*, 2025, doi: 10.1142/S2301385025440054.

[3] L. Zhou, M. Jing, L. Zhong, Z. Yu, and X. Zeng, "Task-binding based branch-and-bound algorithm for noc mapping," in *2012 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2012, pp. 648–651.

[4] T. Okubo and M. Takahashi, "Simultaneous optimization of task allocation and path planning using mixed-integer programming for time and capacity constrained multi-agent pickup and delivery," in *2022 22nd International Conference on Control, Automation and Systems (ICCAS)*. IEEE, 2022, pp. 1088–1093.

[5] S. Wang, Y. Liu, Y. Qiu, and J. Zhou, "Consensus-based decentralized task allocation for multi-agent systems and simultaneous multi-agent tasks," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 12 593–12 600, 2022.

[6] M. Braquet and E. Bakolas, "Greedy decentralized auction-based task allocation for multi-agent systems," *IFAC-PapersOnLine*, vol. 54, no. 20, pp. 675–680, 2021.

[7] Y. Wang, S. Ren, and C. Wang, "Multi-uav multi-task allocation based on cognitive state-based hybrid contract net protocol," in *2025 10th International Conference on Computer and Communication System (ICCCS)*. IEEE, 2025, pp. 921–926.

[8] A. Maoudj, B. Bouzouia, A. Hentout, A. Kouider, and R. Toumi, "Distributed multi-agent approach based on priority rules and genetic algorithm for tasks scheduling in multi-robot cells," in *IECON 2016-42nd Annual Conference of the IEEE Industrial Electronics Society*. IEEE, 2016, pp. 692–697.

[9] J. Xie and C.-C. Liu, "Multi-agent systems and their applications," *Journal of International Council on Electrical Engineering*, vol. 7, no. 1, pp. 188–197, 2017.

[10] K. Leahy, D. Zhou, C.-I. Vasile, K. Oikonomopoulos, M. Schwager, and C. Belta, "Persistent surveillance for unmanned aerial vehicles subject to charging and temporal logic constraints," *Autonomous Robots*, vol. 40, no. 8, pp. 1363–1378, 2016.

[11] S. Xu, X. Luo, Y. Huang, L. Leng, R. Liu, and C. Liu, "Nl2hltl2plan: Scaling up natural language understanding for multi-robots through hierarchical temporal logic task specifications," *IEEE Robotics and Automation Letters*, pp. 1–8, 2025.

[12] C. Baier and J.-P. Katoen, *Principles of model checking*. MIT press, 2008.

[13] S. L. Smith, J. Tůmová, C. Belta, and D. Rus, "Optimal path planning under temporal logic constraints," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2010, pp. 3288–3293.

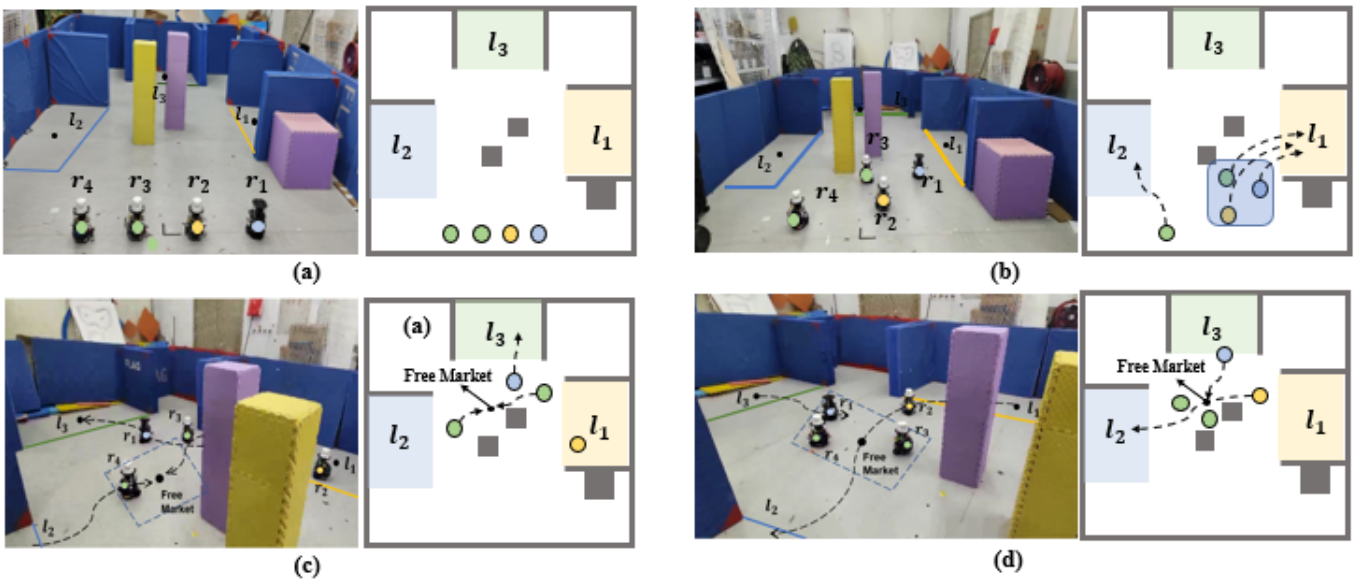

Fig. 10. The dashed lines indicate the robot trajectories. In (a), l_1 to l_3 represent the areas of interest. There are three kinds of robots: $\{r_1\}$, $\{r_2\}$ and $\{r_3, r_4\}$. In (b)-(d), the robots are executing their subtasks.

[14] A. Ulusoy, S. L. Smith, X. C. Ding, C. Belta, and D. Rus, "Optimality and robustness in multi-robot path planning with temporal logic constraints," *The International Journal of Robotics Research*, vol. 32, no. 8, pp. 889–911, 2013.

[15] Y. Kantaros and M. M. Zavlanos, "Sampling-based optimal control synthesis for multirobot systems under global temporal tasks," *IEEE Transactions on Automatic Control*, vol. 64, no. 5, pp. 1916–1931, 2018.

[16] A. M. Jones, K. Leahy, C. Vasile, S. Sadraddini, Z. Serlin, R. Tron, and C. Belta, "Scratchs: Scalable and robust algorithms for task-based coordination from high-level specifications," in *The International Symposium of Robotics Research*. Springer, 2019, pp. 224–241.

[17] Z. Chen and Z. Kan, "Real-time reactive task allocation and planning of large heterogeneous multi-robot systems with temporal logic specifications," *The International Journal of Robotics Research*, vol. 44, no. 4, pp. 640–664, 2024.

[18] Y. Kantaros and M. M. Zavlanos, "Stylus*: A temporal logic optimal control synthesis algorithm for large-scale multi-robot systems," *The International Journal of Robotics Research*, vol. 39, no. 7, pp. 812–836, 2020.

[19] Z. Chen, Z. Zhou, S. Wang, J. Li, and Z. Kan, "Fast temporal logic mission planning of multiple robots: A planning decision tree approach," *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6146–6153, 2024.

[20] P. Schillinger, M. Bürger, and D. V. Dimarogonas, "Decomposition of finite ltl specifications for efficient multi-agent planning," in *Distributed Autonomous Robotic Systems: The 13th International Symposium*. Springer, 2018, pp. 253–267.

[21] ——, "Simultaneous task allocation and planning for temporal logic goals in heterogeneous multi-robot systems," *The International Journal of Robotics Research*, vol. 37, no. 7, pp. 818–838, 2018.

[22] J. Liu, F. Li, X. Jin, and Y. Tang, "Distributed task allocation for multi-agent systems: A submodular optimization approach," *arXiv preprint arXiv:2412.02146*, 2024.

[23] Z. Liu, M. Guo, W. Bao, and Z. Li, "Fast and adaptive multi-agent planning under collaborative temporal logic tasks via poset products," *Research*, vol. 7, p. 0337, 2024.

[24] Y. Zhu *et al.*, "Dexter-llm: Dynamic and explainable coordination of multi-robot systems in unknown environments via large language models," *arXiv preprint arXiv:2508.14387*, 2025.

[25] I. Filippidis, D. V. Dimarogonas, and K. J. Kyriakopoulos, "Decentralized multi-agent control from local ltl specifications," in *2012 IEEE 51st IEEE Conference on Decision and Control (CDC)*, 2012, pp. 6235–6240.

[26] M. Guo and D. V. Dimarogonas, "Multi-agent plan reconfiguration under local ltl specifications," *The International Journal of Robotics Research*, vol. 34, no. 2, pp. 218–235, 2015.

[27] ——, "Distributed plan reconfiguration via knowledge transfer in multi-agent systems under local ltl specifications," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 4304–4309.

[28] D. Tian, H. Fang, Q. Yang, and Y. Wei, "Decentralized motion planning for multiagent collaboration under coupled ltl task specifications," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 52, no. 6, pp. 3602–3611, 2021.

[29] Z. Chen, L. Li, and Z. Kan, "Distributed task allocation and planning under temporal logic and communication constraints," *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6536–6543, 2024.

[30] C. H. Fua and S. S. Ge, "Cobos: Cooperative backoff adaptive scheme for multirobot task allocation," *IEEE Transactions on Robotics*, vol. 21, no. 6, pp. 1168–1178, 2005.

[31] S. Choudhury, J. K. Gupta, M. J. Kochenderfer, D. Sadigh, and J. Bohg, "Dynamic multi-robot task allocation under uncertainty and temporal constraints," *Autonomous Robots*, vol. 46, no. 1, pp. 231–247, 2022.

[32] V. C. Kalempa, L. Piardi, M. Limeira, and A. S. de Oliveira, "Multi-robot preemptive task scheduling with fault recovery: A novel approach to automatic logistics of smart factories," *Sensors*, vol. 21, no. 19, p. 6536, 2021.

[33] C. Belta, B. Yordanov, and E. A. Gol, *Formal methods for discrete-time dynamical systems*. Springer, 2017, vol. 89.

[34] X. Luo, S. Xu, R. Liu, and C. Liu, "Decomposition-based hierarchical task allocation and planning for multi-robots under hierarchical temporal logic specifications," *IEEE Robotics and Automation Letters*, 2024.

[35] Z. Liu, M. Guo, and Z. Li, "Time minimization and online synchronization for multi-agent systems under collaborative temporal logic tasks," *Automatica*, vol. 159, p. 111377, 2024.

[36] C. Bettstetter, H. Hartenstein, and X. Pérez-Costa, "Stochastic properties of the random waypoint mobility model," *Wireless networks*, vol. 10, no. 5, pp. 555–567, 2004.

[37] P. Gastin and D. Oddoux, "Fast ltl to büchi automata translation," in *Computer Aided Verification*, G. Berry, H. Comon, and A. Finkel, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 53–65.